

FastAPI

From Local Model to Production-Ready API

Mosharof Hossain

AI Engineer · Brain Station 23

REST API Basics

Caching

Redis

Load Balancing

REST API Refresher

HTTP Methods · Request/Response cycle

REST (Representational State Transfer) — a client sends an HTTP request → server processes → returns **JSON response**.
FastAPI is built on this pattern, adding automatic validation + docs.

Method	Purpose	Example	Notes
GET	Retrieve data	GET /users	No body — query params only
POST	Create resource	POST /predict	Sends JSON body
PUT	Replace entire resource	PUT /users/1	Replaces all fields
PATCH	Partial update	PATCH /users/1	Only changed fields
DELETE	Remove resource	DELETE /users/1	Deletes the item

Pydantic — The Gatekeeper

Auto-validation, type-safety, and OpenAPI docs — for free

Why Pydantic Matters for AI APIs



Auto-Validation

User sends string where float expected → FastAPI returns 422 before the model even runs



Shape Enforcement

AI models need exact input shapes. Pydantic enforces array dims, string lengths, enums



Free OpenAPI Docs

Your schema becomes interactive /docs automatically — no extra work



Zero Boilerplate

No manual if/else validation code. Declare once, enforced everywhere

Example: Student model + AI prediction input

```
from fastapi import FastAPI
from pydantic import BaseModel

class Student(BaseModel):

    name      : str
    age       : int
    email     : str

class PredictInput(BaseModel):

    text      : str
    max_len   : int = 512
    threshold: float = 0.5

@app.post (/predict)
def predict(data: PredictInput)
    return model(data.text):
```

✓ Invalid input → 422 Unprocessable Entity (auto) — model never runs → saves compute

Why Caching & Load Balancing Matter

The cost of doing nothing at scale

⚠️ WITHOUT — Naive API

- ✗ Response time ↑ as traffic grows
- ✗ Same ML inference repeated 1000x
- ✗ Server crashes under load
- ✗ GPU cost: unbounded
- ✗ One server = single point of failure



✅ WITH — Optimised API

- ✓ Response time ↓ 80%
- ✓ Server cost ↓ 50%
- ✓ Repeated inference cached ⚡
- ✓ Traffic spikes handled
- ✓ One server fails others continue

Caching Concepts — Types & When to Use

Temporarily store results → serve future requests instantly

Caching = saving expensive computation results so future identical requests skip the work entirely. **The cache is your CPU/GPU's memory savings account.**



In-Memory Cache

• Same server (RAM)

e.g. `functools.lru_cache`

Single-server, lightweight



Redis / Distributed

• Shared across servers

e.g. Redis, Memcached

Multi-server, microservices



CDN Cache

• Network edge (global)

e.g. Cloudflare, Akamai

Static assets, global apps



Model / Inference

• AI result cache

e.g. HuggingFace cache,
Redis

LLM/embedding results

When to Cache?

API response rarely changes

✓ Cache

Avoid recomputation

Large DB table reads

✓ Cache

Reduce DB load

ML model inference

✓ Cache

Huge GPU savings

File uploads

✗ Skip

Too dynamic

Real-time stock prices

✗ Skip

Needs freshness

Demo 1 — In-Memory Cache with lru_cache

functools.lru_cache · Zero dependencies · Single server

demo_01_lru_cache.py

```
from fastapi import FastAPI
from functools import lru_cache
import time

app = FastAPI()

@lru_cache (maxsize=50)
def heavy_computation(n: int):
    print (f"Computing {n}...")
    time.sleep (3)
    return {"result": n*n}

@app.get ("/ compute")
def compute(n: int):
    return heavy_computation(n)
```

⚠ Limit: lost on server restart · not shared across instances → use Redis for multi-server

How it works

1

First call /compute?n=10

Cache miss → runs heavy_computation → takes 3 seconds

2

Second call /compute?n=10

Cache hit → returns instantly ↯ no sleep(3) executed

3

Different n value

Cache miss again → computed fresh, stored in RAM

▶ Run it:

```
uvicorn demo_01_lru_cache:app --reload
# GET http://localhost:8000/compute?n=10
```

Demo 2 — Redis Distributed Cache

Shared cache · TTL support · Multi-server ready

demo_02_redis_cache.py

```
from fastapi import FastAPI
import redis, json, time

app = FastAPI()
r = redis.Redis(host="localhost", port=6379)

@app.get("/expensive")
def get_data(q: str):
    key = f"data:{q}"
    cached = r.get(key)
    if cached:
        return json.loads(cached)

    time.sleep(3) # simulate ML inference
    result = {"query": q, "response": f"Done {q}"}
    r.setex(key, 60, json.dumps(result)) # TTL 60s
    return result
```

Setup & Key Concepts

```
# Install
pip install redis fastapi uvicorn

# Start Redis (Docker)
docker run -p 6379:6379 redis

uvicorn demo_02_redis_cache:app --reload
```

`r.get(key)`

Check if result exists in Redis

`r.setex(key, 60, val)`

Store result, auto-expire after 60s (TTL)

`json.dumps/loads`

Serialize Python dict ↔ JSON string

Cache key

"data:{q}" — unique per query string

✓ Survives restarts · Shared by all servers · TTL auto-expires stale data

Demo 3 — AI Model Inference Cache

Never run the same text through the model twice

demo_03_model_cache.py

```
from fastapi import FastAPI
from transformers import pipeline
import redis, json, hashlib

app = FastAPI()
r = redis.Redis(host="localhost", port=6379)
model = pipeline("sentiment-analysis")

def make_key(text: str) -> str:
    hash = hashlib.md5(text.encode()).hexdigest()
    return "model:" + hash

@app.post("/analyze")
def analyze(text: str):
    key = make_key(text)
    cached = r.get(key)
    if cached: return json.loads(cached)
    result = model(text) # GPU call - expensive!
    r.setex(key, 300, json.dumps(result)) # 5 min
    return result
```

Request Flow

- 1 Request arrives**
"I love FastAPI!"
- 2 MD5 hash key**
"model:a3f4..." — unique fingerprint
- 3 Redis lookup**
Hit? → return instantly (0ms)
- 4 Cache miss**
Run model() → GPU inference (200–500ms)
- 5 Store in Redis**
TTL 300s → next same request is free

⚡ Same sentence repeated 1000x
→ 999 GPU calls saved → ~\$0 marginal cost

Cache Invalidation Strategies

Caching is easy — knowing when to expire is hard

"There are only two hard things in computer science: cache invalidation and naming things." — Phil Karlton



TTL (Time to Live)

```
r.setex(key, 60, value)
```

Auto-expire after N seconds. Simplest strategy.

- ✓ Zero manual work
- ✗ Stale data until expiry
- ✓ Guaranteed freshness after TTL



Manual Invalidation

```
r.delete(key)
```

Delete cache entry when underlying data changes.

- ✓ Always fresh
- ✗ Easy to forget on updates
- ✓ Precise control



Versioned Keys

```
"user_v2:123"
```

Include version in key — old keys silently become unused.

- ✓ Safe rollbacks
- ✗ Old keys accumulate
- ✓ No delete needed



Write-Through

```
update DB + cache together
```

Every write updates cache AND DB simultaneously.

- ✓ Always consistent
- ✗ Slower writes
- ✓ No stale reads

💡 Rule of thumb: TTL for read-heavy AI results (5–60 min) · Manual invalidation for user profile data

Load Balancing — Spreading the Burden

Distribute requests across servers · no single point of failure

Load Balancing distributes incoming HTTP requests across multiple server instances so no single server is overwhelmed. It provides **high availability, horizontal scalability, and fault tolerance**.

 *Analogy: Bank with multiple tellers. Each customer (request) goes to the next free teller (server).*

Load Balancing Algorithms

Round Robin	Rotate requests evenly across all servers	Equal-capacity servers
Least Connections	Send to server with fewest active requests	Mixed / variable workloads
IP Hash	Same client IP always goes to same server	Session-based consistency
Weighted	Assign more traffic to stronger servers	Uneven hardware capacity
Random	Random server selection	Testing / low traffic

Demo 4 — Nginx Load Balancer (Local)

Two FastAPI instances · Nginx distributes traffic

nginx.conf

```
events {}
http {
    upstream fastapi_servers {
        server 127.0.0.1:8000;
        server 127.0.0.1:8001;
    }
    server {
        listen 8080;
        location / {
            proxy_pass http://fastapi_servers;
        }
    }
}
```

Run commands

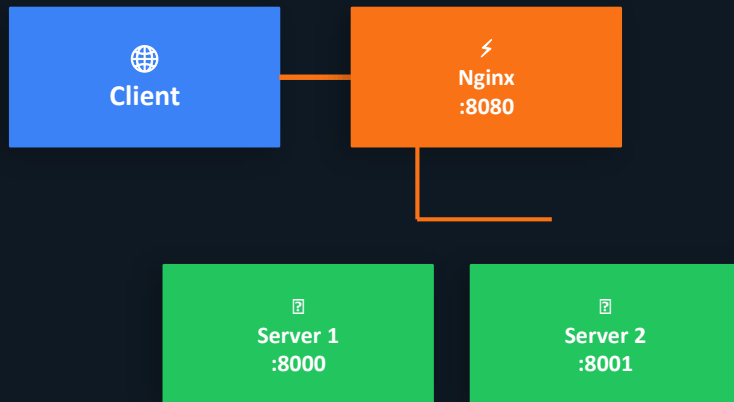
```
# Terminal 1
uvicorn demo_04_lb:app --port 8000

# Terminal 2
uvicorn demo_04_lb:app --port 8001

# Terminal 3 — start nginx
nginx -c $(pwd)/nginx.conf

# Test load balancing
curl http://localhost:8080/info
```

Traffic Flow



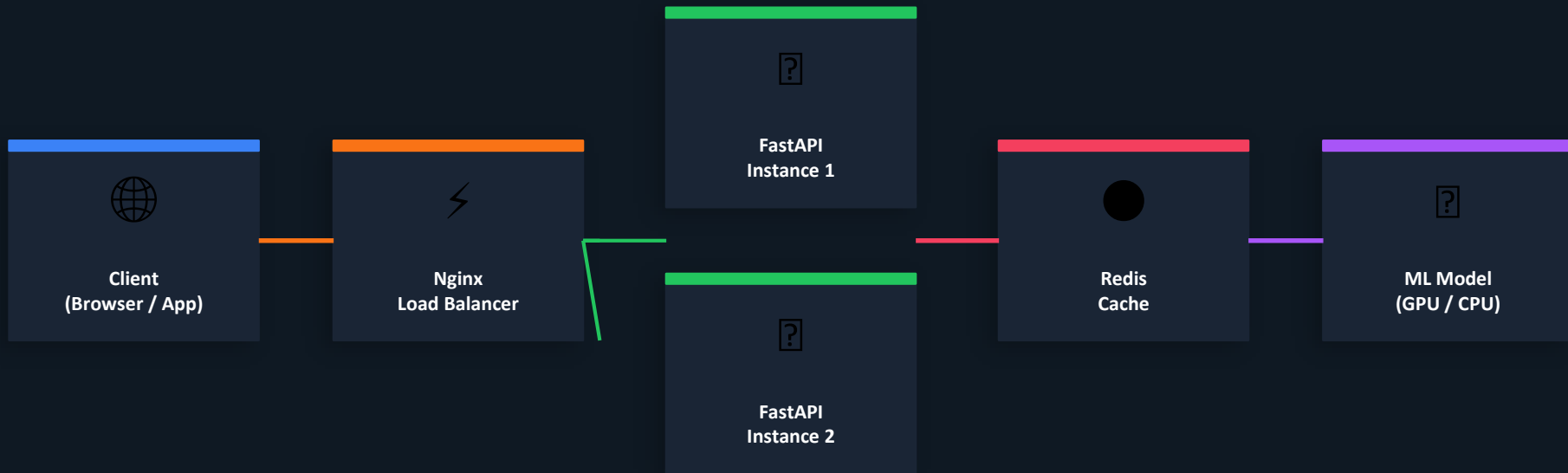
Request 1 → Server 1
Request 2 → Server 2
Request 3 → Server 1
(Round Robin)

Why this matters for ML APIs:

- Each server handles half the GPU load
- Server 1 crash → Server 2 keeps running
- Same pattern used in AWS / GCP production

Complete Production Architecture

Caching + Load Balancing + Pydantic working together



① Validate

Pydantic
422 on bad input

② Distribute

Nginx
Round Robin

③ Cache Check

Redis lookup
Hit → instant return

④ Cache Miss

Call ML model
Store result in Redis

Real-World Scenarios

When to apply each technique



ML Inference API

Cache: Cache model output by input hash (TTL: 5–60 min)

LB: Multiple inference servers — GPU per instance



Search Service

Cache: Cache top 100 queries — 80% of traffic is repeats

LB: Distribute user traffic across shards



E-Commerce

Cache: Cache product details, prices (TTL: 1 min)

LB: Split traffic across regions (geo LB)



Chatbots / LLM API

Cache: Cache embeddings + common prompts (TTL: 30 min)

LB: Scale model inference nodes horizontally

Summary — What We Built Today



FastAPI + Pydantic

REST API basics, HTTP methods, auto-validation, 422 errors, OpenAPI docs



In-Memory Cache

lru_cache — zero deps, single server, instant results for repeated calls



Redis Cache

Distributed cache, TTL, survives restarts, shared across all server instances



Model Inference Cache

MD5 hash key per input — never GPU-compute the same text twice



Cache Invalidation

TTL, manual delete, versioned keys, write-through — design strategy early



Load Balancing

Nginx / Docker Compose — round robin, least connections, fault tolerance